Programmeerproject 1: Verslag “Duck Hunt Deluxe”

Nassihi Tariq

Tariq.Nassihi@vub.be

Bachelor Computerwetenschappen

Fase 2

Rolnummer: 0604643
Academiejaar: 2023-2024

Inhoudsopgave

1. Introductie 3
2. ADT's 4
2.1 Positie ADT 4
2.2 Vogel ADT 5
2.3 Level ADT 6
2.4 Teken ADT 7
2.5 Spel ADT 9
2.6 Wapen-ADT 10
2.7 Levens-ADT 11
2.8 Power-up 11
3. Afhankelijkheidsdiagram 12
4. Planning 13

1. Introductie

Dit document is een verslag dat beschrijft hoe ik de tweede fase van het Duck Hunt spel heb geïmplementeerd. Dit spel maakt deel uit van de cursus " Programmeerproject 1". In deze fase breid ik het spel uit met de volgende functionaliteiten:

1) Drie nieuwe vogels zijn toegevoegd: kraai, condor en buizerd. Ook zijn er een nieuw types valse doelwit toegevoegd.
2) Er zijn drie nieuwe wapens geïntroduceerd: het waterpistool, de turbogeweer en een net.
3) Het spel bevat nu minstens drie levels, elk met hun eigen configuratie van obstakels en rondes.
4) Alle soorten obstakels, waaronder bomen, struiken en stenen, zijn opgenomen in het spel.
5) Spelers hebben de mogelijkheid om het spel te herstarten wanneer ze geen levens meer hebben.
6) Twee nieuwe power-ups zijn toegevoegd: bevriezing en chaos.

Om het spel te starten, moet je op run drukken in het document "spel- opstarten.rkt”.

De rest van dit document is als volgt opgebouwd: Deel 2 geeft een overzicht van de ADT's die ik heb geïmplementeerd. Bij elk ADT leg ik welke operaties er zijn (inclusief de signaturen) en hoe elk ADT helpt voor Duck Hunt te kunnen implementeren. In deel 3 is er een visuele representatie van alle ADT’s van het project, en toont aan welke ADT’s verder bouwen op andere ADT’s. Dit wordt in een afhankelijkheidsdiagram getoond. Het laatste deel bevat een tijdschema dat beschrijft hoeveel tijd ik aan de adt heb besteed.

2. ADT's

Dit deel behandelt de ADT's die ik heb gebruikt om het spel te implementeren. Eerst wordt het doel van elke ADT uitgelegd, daarna worden de bijbehorende operaties getoond.

2.1 Positie ADT

Het positie ADT is een abstractie voor het bijhouden van een positie in een tweedimensionale omgeving (punt met x- en y-coördinaat). Het zijn andere ADT's die deze ADT omvatten, zodat een positie kan worden gevolgd. Het Positie ADT bevat de volgende operaties.

- **Maak-positie:** het is de constructor (aanmaakoperatie) die een positie creëert met 2 gegeven getallen en deze positie stelt een punt voor in de spelwereld.
- **x en y**: dit zijn operaties die de x- en y-coördinaten van een positie geven.
- **x! en y!** dit zijn operaties die de x- en y-coördinaten van een positie veranderen met een ander gegeven getal.
- **gelijk?**: het is een predicaat die 2 posities vergelijkt, gegeven het soort wapen en vogel. Het controleert of de afstand tussen de 2 posities binnen het bereik van het gebruikte wapen ligt.
- **Achter-obstakel?:** predicaat dat zegt of de positie zich achter een obstakel bevindt (obstakel = positielijst) gegeven het level en de soort vogel.
- **Beweeg-naar-steen!:** Een functie die de beweging van een vogel (kraai) naar een steen beheert, rekening houdend met de posities van de steen en de vogel.

Naam	**Signatuur**
Maak-positie (constructor)	(number, number → Positie)
X	number
Y	number
x!	(number → Ø)
y!	(number → Ø)
gelijk?	(Positie, symbool,symbool → boolean)
Achter-obstakel?	(number, symbool → bool)
beweeg-naar-steen!	(positie, vogel → Ø)

2.2 Vogel ADT

Het " vogel " ADT dient om de eigenschappen van een individuele vogel in het Duck Hunt spel te modelleren.
Het vogel ADT bevat de volgende operaties:

- Bij het maken van een nieuw vogel worden verschillende variabelen geïnitialiseerd, zoals de positie, type (echte vogel of valse doelwit), soort, levens, als de vogel verstopt is of niet, snelheid, x en y richting, als het met water is geraakt, als het achter een steen mag gaan, aantal randen voordat het zal vluchten en zijn verstopt-tijd van elke vogel opvragen.
- **Maak-eend:** het is de constructor die een vogel creëert met een gegeven boolean (echte eend of valse doelwit = type) en het soort vogel.
- **Set-iets!:** wordt gebruikt om de waarden van verschillende variabelen te wijzigen op basis van het symbool dat wordt doorgegeven als argument, waarbij elk symbool overeenkomt met een specifieke variabel.
- **Verstopt!:** functie voor het omkeren van de waarde van de variabele verstopt?.
- **random-richting!:** Dit functie instelt een willekeurige richting (links of rechts voor de x-as, omhoog of omlaag voor de y-as) gebaseerd op de parameter.
- **rand-voor-vluchten!:** Dit functie vermindert de resterende stappen voor vluchten en wijzigt ook de snelheid van het vogel voor het vluchten.
- **Snelheid!:** functie die de snelheid wijzigt op een bepaalde waarde tijd als tijd niet negatief is, anders wordt de snelheid ingesteld op 0.

Naam	**Signatuur**
Maak-eend	(bool → eend)
Type	symbool
Soort	symbool
Water?	Boolean
levens	Levens
positie	Positie
Verstopt?	boolean
snelheid	number
Steen?	boolean
rand-voor-vluchten	Number v symbool
Richting-x	Operator
Richting-y	Operator
verstopt-tijd	number
set-iets!	(symbool, (boolean v symbool v operator → eend)
verstopt!	(Ø→Ø)
rand-voor-vluchten!	(Ø → (number , symbool))
random-richting!	(symbool → Ø)
Snelheid!	(number → Ø)

2.3 Level ADT

Deze ADT biedt verschillende functies voor het bijhouden van tijd, het tellen van vogels, het bijwerken van het level en het beheren van de vogel- en doodlijsten.

- Bij het maken van een nieuw level worden verschillende variabelen geïnitialiseerd, zoals vogel-tijd, vogel-counter, dood-tijd, valse-tijd, vogel-max, vogel-lijst en dood-lijst. Deze variabelen houden de status van het level bij, zoals de tijd sinds de laatste vogel verscheen, het aantal vogels dat is verschenen, etc.
- **nieuw-vogel! :** wordt aangeroepen om een nieuwe vogel of power-up aan het level toe te voegen. Deze functie creëert een nieuwe vogel of power-up en voegt deze toe aan de lijst van vogels in het level.
- **update!:** Het wordt gebruikt om het level bij te werken op basis van de verstreken tijd. Deze functie verhoogt de tijd dat vogels zijn verschenen, controleert of er nieuwe vogels moeten worden toegevoegd, en controleert of de tijd dat een vogel dood is al lang genoeg is om te worden verwijderd.
- **vogel-lijst!:** wordt gebruikt om te controleren of het tijd is om een nieuwe vogel aan het level toe te voegen en of het tijd is om hem uit het level te verwijderen. Deze functie wordt ook gebruikt om de overgang naar volgende rondes en levens te beheren.
- **Nieuwe-posities!:** Deze functie werkt de positie van objecten bij op basis van de verstreken tijd en controleert of ze het scherm verlaten, waarbij ze omkeren als ze dat doen of als ze moeten vluchten.
- **geraakt?:** controleert of het kruishaar een vogel raakt en past de status van de vogel aan.
- **verstopt!:** wordt gebruikt om de status (verstopt is of niet.) van een vogelobject te controleren en wijzigen .
- **Vlucht** :deze functie controleer hoeveel vogels in de lijst er momenteel in de vluchtmodus zijn.
- **Reset**: Deze functie wordt gebruikt om het spel te resetten en terug te keren naar de initiële toestand.
- **Vogel-max!:** dit wordt gebruikt om het aantal vogels en het tijd tussen 2 vogels te verdubbelen.
- **Continue!:** deze functie wijzigt de variabel continue om door te gaan naar de volgende functie.

Naam	**Signatuur**
Maak-level	(number, number → level)
Update!	(number, teken,power-up → symbool)
Nieuw-vogel!	(boolean (symbool v power-up),power-up →∅)
Nieuwe-posities!	(teken ,number, number →∅)
geraakt?	(wapen, teken →symbool v power-up)
verstopt!	(vogel, teken, lijst, number, tile ,lijst→∅)
Vlucht	(∅→ number)
Reset	(∅→∅)
Vogel-max!	(operator→∅)
Level	number
Ronde	number
Vogel-lijst	lijst
Continue!	(∅→∅)
Dood-lijst	lijst
Vogel-lijst!	(teken, power-up→∅ v symbool)

2.4 Teken ADT

Het Teken ADT wordt gebruikt om de spelsituatie te tekenen op scherm met behulp van de grafische bibliotheek die we mogen gebruiken. Het Teken ADT bevat de volgende operaties:

- **Maak-teken**: Een venster wordt gemaakt met opgegeven dimensies en titel. Achtergrond, vogel, wapen , menu lagen worden aangemaakt binnen het venster;
- **teken!**: Deze functie tekent de objecten en retourneert een boolean die aangeeft of er veranderingen zijn aangebracht.
- **Vogel-tile-lijst!**: Deze functie tekent de vogels op basis van de lijst van vogelobjecten die zijn doorgegeven.
- **Teken-crosshair!:** deze functie teken de crosshair gegeven een wapen-object.
- **Nieuw-map**: deze functie wordt gebruikt om de juiste achtergrond te tekenen voor elke level.
- **Teken-pw/herhaaltijd/wapen-tekst**: Functies om respectievelijk de indicatie van power-ups, herhaaltijd en de soorten wapen te tekenen.
- **Update-teken!:** update het tekening van alle objecten in het level.
- **Nieuw-tile-lijst!**: wijzigt de tile-lijst met de opgegeven stukken.
- **Verander-crosshair!**: deze functie dient om de tekening van het crosshair te wijzigen, afhankelijk van het type wapen dat wordt doorgegeven, en het nieuwe crosshair tekenen.
- **Update-crosshair!:** update de positie van het crosshair.
- Interactie met de **muis/keyboard:** Callbackfuncties zijn ingesteld om de positie van het kruishaar bij te werken bij een klik met de linkermuisknop, om een nieuw leven te starten wanneer op "Return" wordt gedrukt, en om het spel te herstarten bij het indrukken van "R".
- **Verwijder-tile!:** Deze functie verwijdert een vogel-object uit het venster.
- **Verwijder-pw!:** verwijder de gegeven power-up.
- **Gevlucht?**: Deze functie geeft aan of de opgegeven tile buiten het scherm ligt.
- Game loop instellen: **set-game-loop!,** deze functie stelt de updatefunctie in voor het venster en **set-teken-loop!,** die de updatefunctie is voor alles wat te maken heeft met tekenen.
- **Verwijder-dood!:** functie om dode objecten te verwijderen.
- **Nieuw-current!:** functie om naar de volgende tekening in een tile-sequence te overgaan.
- **Teken-terug**: teken terug een vogel nadat die verstopt was.
- **vogel-tile->dood-tile!:** verandert een vogel-tile naar een dood-tile als de vogel geraakt was.
- **Update-menu!:** update de gegevens die we op de scherm zien.
- **Verwijder-vogels:** functie die alle overige vogels in een level verwijdert.
- **Verwijder-teken**: Verwijdert alles wat zich in het venster bevindt, behalve de score.

Naam	**Signatuur**
Maak- Tekening	(number, number→ tekening)
Teken-object!	(tile, object→ Ø)
vogel-tile-lijst!	(lijst→ Ø)
teken-crosshair!	(wapen→ Ø)
Verwijder-tile!	(tile→ Ø)
set-game-loop!	((number→ Ø) → Ø)
Teken-terug	(vogel→ Ø)
vogel-tile->dood-tile!	(tile, vogel→ tile)
Update-menu!	((symbool, symbool, boolean, symbool ,symbool , boolean, boolean , symbool→ Ø)
Nieuw-map	(number→ Ø)
Gevlucht?	(tile→ boolean)
Verander-crosshair!	(symbool, wapen → Ø)
Nieuw-tile-lijst!	(lijst, lijst → Ø)
Verwijder-vogels!	(Ø→Ø)
Update-teken!	(Ø→Ø)
Update-crosshair!	(number, number, symbool→Ø)
Verwijder-dood!	(lijst→Ø)
Set-teken-loop!	((Ø→Ø)→ Ø)
Nieuw-current!	(number→Ø)
Verwijder-pw!	(power-up →Ø)
Teken-pw	(Ø→Ø)
Teken-herhaaltijd	(Ø→Ø)
Verwijder-teken	(Ø→Ø)
Vogel-tile-lijst	lijst
Teken!	(lijst, lijst→boolean)
set-mouse-click-loop!	((symbool, symbool,number,number→Ø)→ Ø)
set-key-callback!	((symbool, char→Ø)→ Ø)
Teken-wapen-tekst	(Ø→Ø)

2.5 Spel ADT

Het Spel ADT is de centrale ADT die het hele spel start. Het is verantwoordelijk voor het starten van de spelwereld waarop de vogels beginnen te verschijnen. Het Spel ADT bevat de volgende operaties:

- **Maak- spel:** het is de aanmaakoperatie van het Spel ADT.
- **start!:** het is de operatie die het spel opstart. Wanneer deze operatie uitgevoerd wordt zal er een venster aangemaakt worden en zal het spel opgestart worden en kan de speler interageren met het venster voor het spel te spelen.
- **game-loop!:** is verantwoordelijk voor het regelmatig bijwerken van verschillende aspecten van het spel, zoals het controleren van levens, het bijwerken van power-ups en wapens, het berekenen van nieuwe posities van game-elementen,… .
- **teken-loop!:** het aanpassen van de tekeningen van het spel.
- **keyboard-loop!:** is verantwoordelijk voor het afhandelen van toetsenbordgebeurtenissen in het spel. Als de gebruiker op de Return-toets drukt, wordt het huidige niveau voortgezet. Als de gebruiker op de 'r'-toets drukt begin het spel terug opnieuw.
- **mouse-loop!:** om de positie van het kruishaar bij te werken wanneer er met de linkermuisknop wordt geklikt.
- **verander-wapen:** functie om van wapen te wisselen tijdens het spel.
- **nieuw-power-up:** maak nieuwe power-ups nadat ze zijn gebruikt.
- **nieuw-power-up?:** beheert de levenscyclus van power-ups in het spel, zoals hun activering, effect op de gameplay, en herstel naar een inactieve staat.

Naam	**Signatuur**
Maak-spel	(Ø→ Spel)
Start!	(Ø→Ø)
Game-loop!	(number→Ø)
keyboard-loop!	(symbool, char→Ø)
Mouse-loop!	(symbool, symbool,number,number →Ø)
Teken-loop!	(Ø→Ø)
Verander-wapen	(symbool →Ø)
Nieuw-power-up	(Ø→Ø)
Nieuw-power-up	(Ø→Ø)
Teken-ADT	teken
Level-ADT	level
Wapen-ADT	wapen

2.6 Wapen-ADT

Deze ADT definieert een wapenobject in het spel waarmee je op vogels en power-ups kan schieten.

- **maak-wapen:** Dit is de constructorfunctie voor het maken van een nieuw wapenobject. Binnen deze functie worden verschillende eigenschappen van het wapen geïnitialiseerd, zoals de herhaaltijd tussen schoten, het aantal kogels, de huidige wapentijd en de positie van het wapen.

- **wapen-tijd!** : Deze functie verhoogt de huidige wapentijd met de opgegeven tijdseenheid. Het wordt gebruikt om de tijd bij te werken sinds het laatste schot.

- **schieten?:** Dit is een predicaatfunctie die controleert of het wapen klaar is om te schieten. Het controleert of de huidige wapentijd groter is dan of gelijk aan de herhaaltijd tussen schoten.

- **reset-tijd!:** Deze functie reset de wapentijd naar nul, na het afvuren van een schot.

- **reset-positie**: Reset de crosshair naar zijn oorspronkelijke positie."

Naam	**Signatuur**
Maak-wapen	(symbool →Ø))
Wapen-tijd!	(number→Ø)
Schieten?	(Ø→boolean)
Reset-tijd!	(Ø→Ø)
Wapen-tijd	number
Herhaal-tijd	number
reset-positie	(Ø→Ø)
positie	positie
soort	symbool
kogeks	number

2.7 Levens-ADT

Deze ADT creëert een levensobject om de levens van een object bij te houden en te beheren.

- **maak-levens:** Dit is de constructorfunctie voor het maken van een levensobject gegeven het het soort object.

- **verminder-levens!:** Dit functie wordt gebruikt om het aantal levens te verminderen met een opgegeven waarde x. Het past de levens aan en controleert of het aantal levens nul of minder is. Als dat het geval is, wordt het levensobject gemarkeerd als "dood".

- **Vermeerder-levens!:** vermeerder het aantal levens met de gegeven aantal.

Naam	**Signatuur**
Maak-levens	((symbool v number)→∅)
Verminder-levens!	(number→ (number v symbool))
Vermeerder-levens!	(number→∅)

2.8 Power-up

De Power-up ADT vertegenwoordigt verschillende eigenschappen en operaties die horen bij een power-up.

- De functie **maak-power-up** neemt een argument soort dat het type power-up aangeeft en maakt een nieuw power-up-object. Dit object heeft verschillende variables zoals power-up-tijd, actief?, levens, geactiveerd?, snelheid, en positie.
- **Activeer-pw!**: functie die wordt aangeroepen wanneer een bevriezing power-up wordt geactiveerd. Het past de snelheid van vogels aan in de spelwereld, afhankelijk van de gegeven operator.

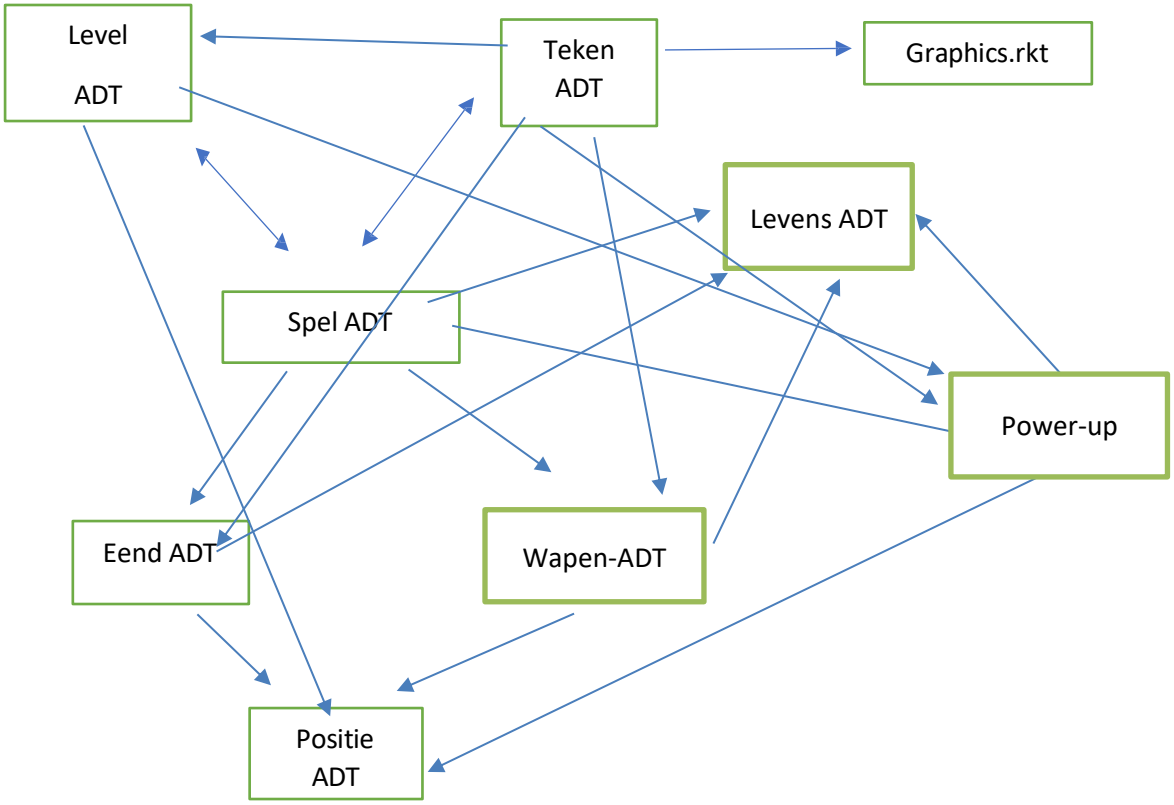
- **Set-iets!:** wordt gebruikt om de waarden van verschillende variabelen te wijzigen op basis van het symbool dat wordt doorgegeven als argument, waarbij elk symbool overeenkomt met een specifieke variabele.
- **Update!:** wordt aangeroepen om de status van de power-up te updaten na verloop van tijd, bijvoorbeeld om te controleren of de power-up nog actief is of dat de tijdslimiet is bereikt.

Naam	**Signatuur**
Maak-power-up	(symbool→∅)
Activeer-pw!	(operator, lijst→∅)
Set-iets!	(symbool, (symbool v number v boolean)→∅)
Update!	(number→∅)
Soort	Symbool
type	symbool
Actief?	symbool
levens	levens
Verstopt?	boolean
positie	positie
Geactiveerd?	boolean
snelheid	number

3. Afhankelijkheidsdiagram

In dit deel laat ik het afhankelijkheidsdiagram zien van mijn implementatie van Duck Hunt. In het spel zijn er in totaal 7 ADT's. Een pijl tussen 2 ADT's betekent dat de ADT afhankelijk is van de andere ADT waarnaar wordt verwezen. De belangrijkste afhankelijkheden zijn hieronder opgesomd.

- Het Teken ADT is het enige ADT dat gebruikt maakt van de functionaliteit van de grafische bibliotheek (graphics.rk). Dus het Teken ADT afhankelijk van de bibliotheek.
- Het Teken ADT hangt zelf ook af van het Spel ADT, omdat het informatie nodig heeft van het Spel ADT om de huidige situatie van het spel te tekenen.
- Het Spel ADT is afhankelijk van het eend ADT omdat het spelelementen bevat die aangestuurd worden vanuit het Spel ADT.
- het eend, wapen, power-up en level ADT is afhankelijk van het Positie ADT omdat ze posities bijhouden



4. Planning

- Week 26: Voorstudie gemaakt
- Week 29: nieuwe soorten vogels (ADT voor buizerd, kraai en condor)
- Week 30: die nieuwe vogels in het spel doen bewegen + juiste interacties voor elke vogel
- Week 31: ADT ‘s voor de nieuwe wapens.
- Week 32-33 de verschillende levels en de mogelijkheid om het spel te herstarten.
- Week 34: Power-up ADT implementeren.
- Week 35 : spel finaliseren
- Week 36: verslag maken